

省下 Codex 额度的"邪修" workflow

—— 网页版 GPT-6 Pro 负责"想", Codex 只负责"干" ——

来源: 数字生命卡兹克 @Khazix0918 X 文章《分享一个大幅节省Codex额度的邪修方法, 不要浪费了你的ChatGPT Pro 会员》(2026-09-17, x.com/i/article/2100430998769131520) | 拾遗任务 515f0276 | 配实验: 只读 MCP Server 全部真跑通 (10/10 PASS)

30 秒版

订阅费买的是**两条互不通气的额度管道**: Codex 的编码额度、ChatGPT 网页版的 Pro 对话额度。

旧干法: 让 Codex 里的最贵模型又想又干 → 一次规划烧掉周额度 10%。邪修干法: 把"想"搬到几乎免费的网页版 GPT-6 Pro 上 (给它挂一个**只读的生产数据 MCP Server**, 让它看得见真实业务), 方案出来点一下 **"添加到 Codex"**, Codex 只做执行——每任务只烧 4%。同一账号周吞吐 **7 件 → 25 件 (3.6 倍)**。

疑问即目录 (12 问)

1. "邪修"到底是什么? 省的是哪笔钱?
2. 为什么网页版 Pro 额度和 Codex 额度是分开的?
3. "代码/生产数据/PR历史"三句话为什么成立?
4. 为什么 GPT Pro 看不到生产数据? 插件底层是什么?
5. "只读、最小权限、可审计"分别怎么落实?
6. 为什么必须接 OAuth (飞书) 鉴权?
7. "添加到 Codex"按钮到底做了什么?
8. 这笔额度账到底怎么算? 能省多少?
9. 为什么官方通道无风险、三方桥接有风险?
10. 和我已学的"混合模型分工流水线"什么关系?
11. 这套思路能平移到 Claude Code 等其他工具吗?
12. 我没有 \$200 Pro 会员, 这方法对我有什么用?

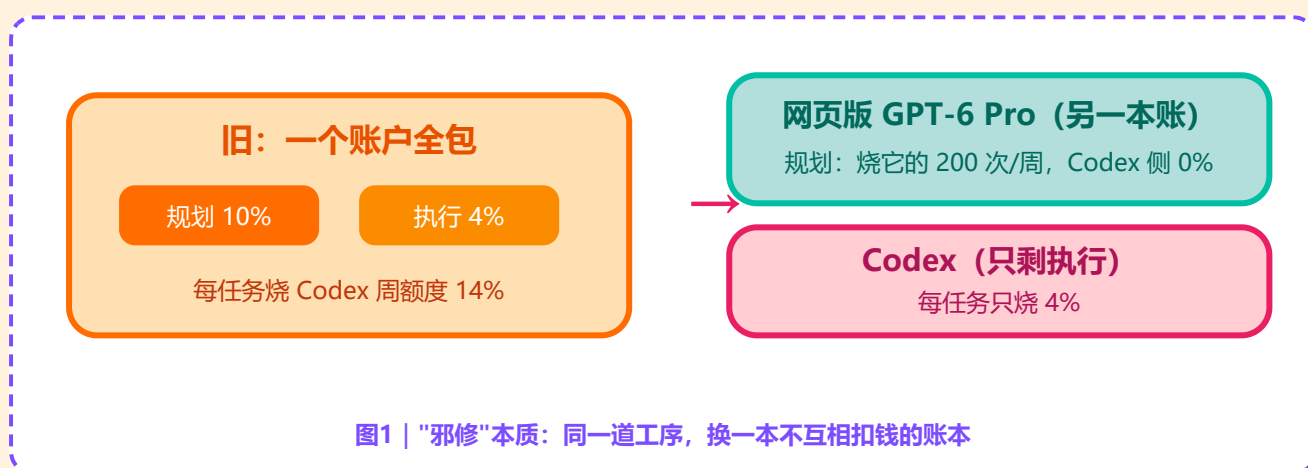
Q1

"邪修"到底是什么？省的是哪笔钱？

不是漏洞、不是外挂、不碰任何反代——它只是把一次开发任务的**"想"和"干"两道工序拆开，分别记到两个账户的账本上。**

一次"给产品降本"的任务 = 读数据 → 分析规划（贵）→ 写代码执行（相对便宜）。旧干法里规划交给 Codex 里的 GPT-6 Astra Ultra，正文原话："Ultra 规划一次就能烧掉 200 刀会员周额度的 10%"——最贵的推理钱花在了"看材料写方案"上。

邪修 = 规划这道工序挪到额度完全独立、每周白送 200 次的网页版 GPT-6 Pro，Codex 只剩下 4% 一次的执行钱。



Q2

为什么网页版 Pro 额度和 Codex 额度是分开的？

因为它们在产品里就是**两个不同的产品面，各配一条配额管道**：网页版 ChatGPT 的"Pro 对话"按**次数**计（每周重置），编码工具 Codex 按**额度池**计（也是每周重置）。订阅费一次买的是这两条管道的**各自额度**，互相不结转、不通兑。

订阅档位	网页版 GPT-6 Pro 对话	Codex 额度	本方案眼中的价值
\$100 / 月	每周 50 次	独立额度池	规划次数少，个人项目够用
\$200 / 月	每周 200 次	独立额度池	正文场景：规划 60 次/周仍有余

大多数人的浪费：让 Codex 又想又干（规划也烧 Codex 账），网页版那 200 次 Pro 对话在旁边吃灰。

类比：健身房年卡里其实含两样东西——器械区（Codex）和私教课（网页版 Pro）。你以为卡只有器械区，天天用最贵的私教器械自己练规划，私教课一节没上。

Q3

"代码 / 生产数据 / PR 历史"三句话为什么成立？

这是全文的方法论内核，三个信息源回答三个不同的问题：

代码

理论上应该怎么运行
(设计意图/规格书)

生产数据

实际上是怎么运行的
(日志/数据库/真实用量)

PR 历史

为什么变成了今天这屎山
(每个决定的当时的理由)

三样一起喂给 GPT Pro，规划才不是纸上谈兵

图2 | 规划的"三视角喂料"：蓝图 × 实况 × 演化史

举例：想"降低 API 成本"。只看代码，AI 会说"加缓存吧"（理论正确）；看到生产数据发现 90% 成本来自 Ultra 规划调用，方案立刻变成"把规划挪走"（实际有效）；再看 PR 历史知道"这里曾经上过缓存又被撤了，因为 stale 数据出过事故"，就不会再提同样的馊主意。**缺任何一路，方案都可能在重蹈覆辙。**

Q4

为什么 GPT Pro 看不到生产数据？插件底层是什么？

网页版 ChatGPT 天然**是沙箱里的聊天员**：它能看 GitHub 插件送来的代码和 PR，但服务器日志、线上数据库在你的机房里，它的手够不着。痛点原话对比："代码告诉 AI 理论上怎么运行，生产数据告诉 AI 实际上怎么运行"——拿不到后者的规划只是猜。

破局点：网页版聊天模式**可以挂插件，而插件的底层正是 MCP（模型上下文协议）**——你自建一个 MCP Server 挂上去，GPT Pro 就多了一双伸进你生产环境的"只读的手"。

	插件 (MCP)	Skill (技能包)
本质	工具接口：一组可调用的函数	方法论文件包：教 Agent 怎么做事的说明书
网页版 ChatGPT	✅ 能挂 （本方案的地基）	❌ 不能调
对应已学概念	[[MCP服务器]]、[[LLM工具调用]]	[[AgentSkills技能包]]

所以文章让 Codex 去"造"的不是 Skill 而是插件——载体选型由**宿主能挂什么**决定。

Q5

"只读、最小权限、可审计"分别怎么落实？

给 AI 开生产数据的口子，安全全压在这六个字上。每一词都有对应的工程手段（实验里逐一验证过）：

原则	技术落点	实验验证（真跑结果）
只读	① SQLite <code>file:...?mode=ro</code> 驱动层物理只读 ② SQL 关键词校验：只放行纯 SELECT	INSERT 被拒： <code>attempt to write a readonly database</code> ；三条写 SQL 全被应用层拦截
最小权限	③ 工具目录只有 2 个白名单工具，没有任何写工具存在 ④ 表名白名单（sqlite_master 也进不来）	<code>write_everything</code> → "未知工具"； sqlite_master → "表不在白名单"
可审计	⑤ 每个请求（含被拒的）追加写 JSONL 审计日志	客户端 8 条请求 = 审计日志 8 行（拒 5），逐条对账

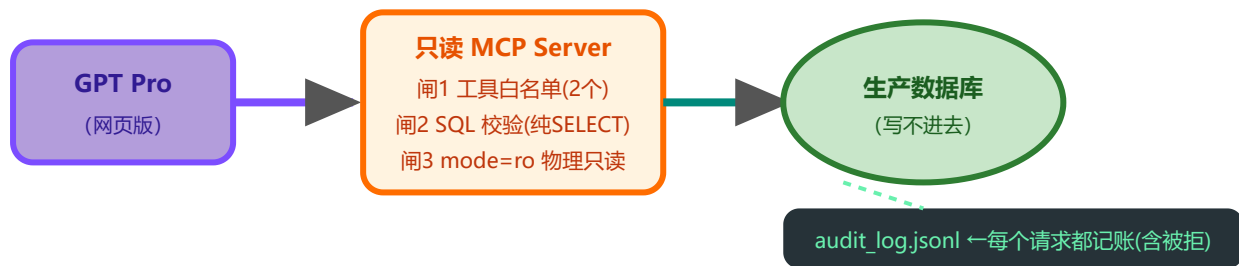


图3 | 三道闸 + 一本账：即使前两闸被绕过，驱动层也写不进；无论成败都留痕

要点：安全要做成"多层闸门"而不是一句提示词——提示词可能被注入绕过，`mode=ro` 是数据库驱动层面的物理事实。

Q6

为什么必须接 OAuth（飞书）鉴权？

MCP Server 一旦挂到网页版，就是一个挂在公网、能读生产库的服务。正文里 Codex 主动提出"需要 OAuth 登录服务"，而公司已有飞书认证中心，于是直接接飞书鉴权——只有作者本人的飞书账号登录后才能用。

- 不做鉴权 = 生产库裸奔：任何知道 Server 地址的人（或被提示注入攻陷的会话）都能借它的工具读你的用户表、成本表。
- 为什么用现成的飞书而不是自建账号体系：认证中心已管着"谁是谁"，复用 = 最小实现 + 权限天然与组织架构对齐。
- 这与已学的 [[OAuth资源发现]] (RFC 9728 远程 MCP 免贴 Key) 是同一命题的两端：那是客户端自动拿凭证，这是服务端用企业 IdP 把住门。

Q7

"添加到 Codex"按钮到底做了什么？

GPT Pro 推理约 40 分钟产出高质量方案后，不用下载 md 文档再手动喂给 Codex——点聊天窗口的 **"添加到 Codex"**，整段对话（含方案全文）作为上下文直接出现在 **Codex 窗口**。这是官方给"网页版 ⇌ 编码端"修的一条 **上下文接力管道**：规划现场的所有推理成果零损耗搬家。

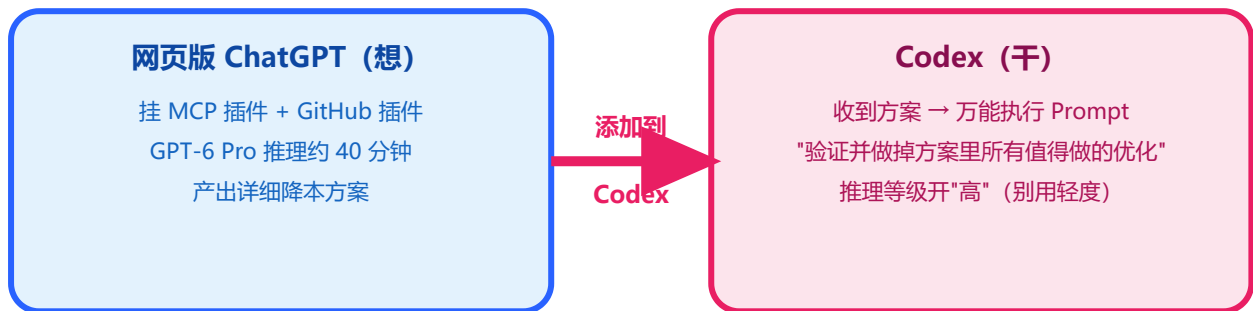


图4 | "添加到 Codex": 规划上下文的一键接力管道 (免下载-上传的手动搬运)

两个执行细节：① 执行 Prompt 是**授权式**的 ("所有值得做的") 而不是逐条点名，把二次决策也交给 Codex；② 推理等级开"高"——轻度档容易反复失败或不验证，省下的小钱会被重试烧回去。

Q8

这笔额度账到底怎么算？能省多少？

用文章给的数字建一个账本模型（实验 step4 真跑输出）：

方案	说明	每任务耗%	周吞吐(任务)
旧 workflow	Ultra 规划+执行全在 Codex	14%	7
邪修 workflow	网页版 Pro 规划(0% Codex)+Codex 执行	4%	25

同一账号每周吞吐提升 3.6 倍；等价于同样的活，Codex 额度只花 1/3.6

业务需求按每周 60 个优化任务算（三账号 \$600/月实况）：

旧 workflow 需要 9 个 \$200 账号轮换，一年 \$21,600

邪修后需要 3 个 \$200 账号轮换，一年 \$7,200

一年省 \$14,400 (省 67%)

网页版侧：每周规划 60 次，\$200 档 200 次/周 够用（余 140），\$100 档 50 次不够

解读：省的大头不是那 10% 规划费本身，而是 **"规划费压在每任务头上"**这个固定开销被归零后，同样的钱能养活的吞吐量翻了 3.6 倍。额度经济学和已学的 [[上下文预算与战略压缩]] 一个道理，只是省的不是 token 是会员费。

Q9

为什么官方通道无风险、三方桥接有风险？

	本方案（官方插件机制）	三方桥接 / 反代
做法	在产品允许的地方挂自己的 MCP 插件	把网页版 Pro 额度用程序"转接"给 Codex 用
本质	扩大自己的可见信息面，账还是各记各的	把 A 产品的额度搬运进 B 产品，绕开计费边界
风控视角	插件调用是正常产品行为，模式与普通用户无异	共享凭证、自动化特征明显，是平台明确打击的账号倒卖/滥用形态
作者原话	"基本无风险，不会降智或触发风控"	"如果被风控了，别找我"，账号安全第一

判别标准一句话：你在用产品已有的能力做新组合（安全），还是在抹平两个产品间的计费边界（危险）。

Q10

和已学的"混合模型分工流水线 / 能力地板选型"什么关系？

它们是同一思想的两个维度：

概念	路由依据	省的是什么	关系
[[混合模型分工流水线]] (paperroute)	任务类型（代码/概念图/3D网格）	单模型包揽的低效与上下文混乱	本篇是它的"账户维度"特例：工序不变，只按"哪本账便宜"分配规划/执行
[[能力地板选型]] (wechat_hermes_notebot)	能力地板（够用就行选最便宜）	按天花板付钱的浪费	规划要顶配（Pro），执行 Astra High 就够——正是"按地板选"的双账户版
[[上下文预算与战略压缩]]	上下文预算	token	同是"预算管理"，一个省窗口、一个省会员费
[[快照契约与不可变只读]] [[CleanRoom只读读取器]]	只读数据交接	数据安全	本篇 MCP 的 mode=ro + 白名单 + 审计正是这两个概念在"给 AI 开生产数据口"场景的落地

Q11

这套思路能平移到 Claude Code 等其他工具吗？

能。把具体产品名抽掉，剩下的是一个**可复用的通用模式：贵模型只读规划 → 便宜档执行 → 规划上下文接力 → 生产数据走只读通道**：

模式零件	Codex 生态	其他生态的对应
独立的"想"额度	网页版 GPT-6 Pro (200 次/周)	任何订阅里被闲置的高端对话额度 (如 Claude.ai 网页版 opus 对话)
只读数据通道	自建只读 MCP Server + 企业 OAuth	Claude Code/其他 Agent 挂同一个只读 MCP (MCP 本就跨宿主)
上下文接力	"添加到 Codex"按钮	方案落成一份 PLAN.md / spec 文件，执行端开工时读取 ([规格书先行SpecDriven])
便宜执行	Codex Astra High (4%/任务)	各家的中档编码模型 + 本地 Agent 循环

其中"上下文接力"在别的生态退化为一个 md 文件——这正好就是已学 [[Agent指令文件AgentMd]] 与 [[规格书先行SpecDriven]] 的用武之地：**规划的产物是文档，执行的输入也是文档，文档就是那条管道。**

Q12

我没有 \$200 Pro 会员，这方法对我有什么用？

- **\$100 档也适用**：正文明说每周 50 次 Pro 对话同样能走这条路；实验算过 60 次/周的需求会不够，但个人项目一周做不了 60 次大规划——**次数够不够，取决于你把"规划"这件事的频次设计得多高。**
- **零成本也能学走两样**：①"三视角喂料" (代码+生产数据+历史决策) 是规划质量方法论，不花一分钱；②"只读最小权限可审计"的 MCP Server 是通用基建，本实验 130 行 Python 就造出来了，挂给任何 Agent 都行。
- **真正的可迁移资产是"分账思维"**：下次要烧贵额度前先问一句——**这道工序能不能记到另一本更便宜/免费的账上？** 能搬就搬，搬不了的才留在贵账户上干。

这个项目给我什么（能力清单）

1. **会算额度账**：拿到任何订阅，先画出它的额度管道图（几条池子、按什么计、多久重置、互不结转），再决定工序怎么摆。
2. **会给 AI 开"生产数据只读口"**：三道闸（工具白名单 / SQL 校验 / mode=ro）+ JSONL 审计，130 行 Python 可复制的模板（见实验）。
3. **会跑"贵想便宜干"双账户 workflow**：规划在网页版挂 MCP+GitHub 插件完成 → 一键/文档接力 → 执行端只干不猜。
4. **会用三视角喂料**：让 AI 做方案前同时给代码（理论）、生产数据（实际）、决策历史（为什么），方案不再纸上谈兵。
5. **会判安全边界**：官方能力组合 = 安全；抹平计费边界的搬运 = 风控雷区。
6. **能落地到自己的项目**：AIHOT 式"降本分析"任务模板——先聚合成本数据（按模型/按端点），再让贵模型出方案，执行端统一上线。

动手实验（全部真实运行，环境 Python 3.14.7 / Windows 11）

主实验：复刻文章核心——"只读、最小权限、可审计"的生产数据 MCP Server（模拟 AIHOT 生产库）。目录：`codex_quota_mcp\exercise\`

1. **造模拟生产库**：`python step1_mock_prod_db.py` —— 目的：还原"线上业务库"当靶子。✅ 预期/实测：`[OK] users=200 行, api_costs=164 行`（含按模型聚合的成本流水，Ultra 单价×25 还原"规划贵"）。
2. **写只读 MCP Server**：`step2_readonly_mcp_server.py` —— 目的：实现三原则。✅ 实现：stdio JSON-RPC（MCP 传输）；闸1 工具目录只有 `get_cost_summary`/`query_table`；闸2 SQL 只放行纯 SELECT；闸3 SQLite `file:...?mode=ro`；每个请求追加 `audit_log.jsonl`。
3. **当客户端逐项验收**：`python step3_test_readonly_mcp.py` ✅ 预期/实测：**10/10 PASS**——

```
[PASS] T1 最小权限-工具目录白名单      tools=['get_cost_summary', 'query_table']
[PASS] T2a 只读放行-成本聚合             {"gpt6-astra-ultra": 31.78 USD ...}
[PASS] T3 拦截写SQL[INSERT/DELETE/DROP] 只允许纯 SELECT（拦截写关键词）
[PASS] T4 驱动层 mode=ro 拦截            attempt to write a readonly database
[PASS] T5 白名单外表/目录外工具拒绝      表不在白名单: sqlite_master / 未知工具
[PASS] T6 可审计-JSONL对账               请求 8 条 = 审计 8 行（其中拒绝 5 条）
```

4. **额度账模拟**：`python step4_quota_calc.py` ✅ 预期/实测：旧 workflow 14%/任务 → 周吞吐 7；邪修 4%/任务 → 周吞吐 **25** (3.6×)；60 任务/周场景一年 **省 \$14,400 (67%)**；\$200 档 200 次 Pro/周规划够用、\$100 档不够。

实验结论：文章的核心安全主张可以在百行级代码里完整落地并被自动化验收——"只读"靠驱动层物理事实而非提示词，"可审计"能做到逐请求对账。

坑与提醒

- **规划别挪走执行**：邪修只省"想"的钱，执行仍要 Codex 认真跑——推理等级开"高"，轻度档反复失败反而更烧。
- **网页版能挂 MCP 插件、不能挂 Skill**：载体选型先问宿主支持什么，别把力气花在挂不上的东西上。
- **数据口开了就要管**：只读 + 白名单 + 审计 + OAuth 四件缺一不可；接入企业现成 IdP（飞书）比自建账号体系省一半工。
- **别碰三方桥接/反代**：省的是小钱，赌的是账号——被风控了没有找回通道。
- **文章数字会过期**：10%/4%/200 次是 2026-09 时点该档位的数，学的是"分账思维"，数字要按当时最新价目重算。